

RAPPORT DATA MINING
Prédiction de Survie sur le Titanic
Année Universitaire 2025-2026

Étudiantes 2ème Année Cycle Ingénieurs
Génie Informatique

JAZOULI KHADIJA
JALLAM SALMA
SIDI MARIEM
TOUOYEM NGOUEGHEU Joyce Fortuna

30 Décembre 2025

Table des matières

1	Cadre Théorique	3
1.1	Contexte du Projet	3
1.2	Pourquoi le Data Mining ?	3
2	Description de la Base de Données	3
3	Processus du Projet	4
3.1	1. Importation des Librairies	4
3.2	2. Chargement des Données	5
3.3	3. Exploration et Nettoyage	5
4	Algorithmes de Prédiction	6
4.1	Logistic Regression	7
4.2	K-Nearest Neighbors (KNN)	7
4.3	Naive Bayes (Gaussian NB)	8
4.4	Decision Tree	9
4.5	Random Forest	9
4.6	Gradient Boosting	9
4.7	Support Vector Machine (SVM)	9
5	Évaluation des Modèles	10
5.1	Résultats des Performances	10
5.2	Analyse des Résultats	10
6	Préprocessing Détaillé	11
6.1	Traitement des Valeurs Manquantes	11
6.2	Standardisation Z-Score	11
6.3	Validation Croisée et Split Train/Test	11
7	Interface de Prédiction	11
8	Conclusion et Perspectives	12
9	Remerciements	12
10	Références	13

1 Cadre Théorique

1.1 Contexte du Projet

Le naufrage du Titanic en 1912 reste l'un des événements les plus tragiques de l'histoire maritime. Sur les 2224 passagers et membres d'équipage à bord, plus de 1500 personnes ont péri dans la catastrophe. Ce projet de Machine Learning vise à prédire la survie des passagers en se basant sur leurs caractéristiques personnelles et contextuelles.

La base de données Titanic est un dataset de référence en apprentissage automatique, contenant des informations telles que l'âge, le sexe, la classe du billet, le tarif payé, le port d'embarquement et la présence de famille à bord. L'objectif est de construire des modèles capables de prédire si un passager a survécu (1) ou non (0).

Ce dataset présente un défi intéressant car la survie dépend de multiples facteurs interconnectés : socio-économiques (classe), démographiques (âge, sexe), familiaux (présence de proches) et contextuels (port d'embarquement).

1.2 Pourquoi le Data Mining ?

Dans un monde où les entreprises et organisations produisent des milliards de données quotidiennement, l'exploitation intelligente de ces informations devient cruciale. Le data mining permet d'extraire des connaissances actionnables à partir de données brutes, révélant des patterns invisibles à l'analyse humaine traditionnelle.

Pour ce projet spécifiquement, le data mining offre plusieurs avantages :

- **Analyse multidimensionnelle** : Prise en compte simultanée de multiples variables influençant la survie
- **Détection de patterns complexes** : Identification de corrélations non intuitives entre variables
- **Prédiction probabiliste** : Estimation de la probabilité de survie pour de nouveaux passagers
- **Comparaison d'algorithmes** : Évaluation objective des performances de différents modèles ML

2 Description de la Base de Données

Le dataset Titanic contient 891 observations (passagers) et 15 variables descriptives. Les colonnes principales utilisées sont :

Variable	Description
survived	Variable cible : 0 = non-survivant, 1 = survivant
pclass	Classe du billet (1=Première, 2=Deuxième, 3=Troisième)
sex	Sexe du passager (male/female)
age	Âge en années (valeurs manquantes : 20%)
sibsp	Nombre de frères/sœurs/conjoints à bord
parch	Nombre de parents/enfants à bord
fare	Prix du billet payé
embarked	Port d'embarquement (C=Chérbourg, Q=Queenstown, S=Southampton)
alone	Voyageait seul (True/False)

Les statistiques descriptives montrent :

- Taux de survie global : 38.4%
- Âge moyen : 29.7 ans (écart-type : 14.5)
- Tarif moyen : 32.2\$ (très dispersé : écart-type 49.7)
- 77% des passagers voyageaient seuls

3 Processus du Projet

Le projet suit la méthodologie CRISP-DM adaptée au Machine Learning :

3.1 1. Importation des Librairies

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Modèles de classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score

# Sauvegarde du modèle
import joblib
import os
```

```
import pickle
import warnings
warnings.filterwarnings("ignore")
```

Listing 1 – Importation des bibliothèques essentielles

3.2 2. Chargement des Données

```
df = sns.load_dataset("titanic")
df.head(10)
```

Listing 2 – Chargement du dataset Titanic

3.3 3. Exploration et Nettoyage

- **Suppression** des colonnes redondantes : deck, embarktown, alive, class, who, adult-male

```
df.drop(columns=[
    "deck",
    "embark_town",
    "alive",
    "class",
    "who",
    "adult_male"
], inplace=True)
```

- **Imputation** des valeurs manquantes : âge (moyenne), embarked (mode)

```
# V r i f i c a t i o n  d e s  v a l e u r s  m a n q u a n t e s
df.tail(10)
df.isnull().sum()

total = len(df)
nb_nan_rows = df.isnull().any(axis=1).sum()
proportion = (nb_nan_rows / total) * 100

print(f"Total lignes : {total}")
print(f"Lignes avec NaN : {nb_nan_rows}")
print(f"Proportion : {proportion:.2f}%")

# D e c i s i o n
if proportion < 5:
    print("    Proportion faible : suppression des lignes avec NaN")
    df = df.dropna()
else:
    print("    Proportion levee : imputation appliquee")

    def clean_missing_values(df):
        df_clean = df.copy()
        strategies = {
            "sex": "mode",
            "embarked": "mode",
            "alone": "mode",
            "pclass": "mode",
```

```

        "sibsp": "median",
        "parch": "median",
        "age": "mean",
        "fare": "mean"
    }

    for col, strategy in strategies.items():
        if col in df_clean.columns and df_clean[col].isnull().sum() > 0:
            if strategy == "mode":
                fill_value = df_clean[col].mode()[0]
            elif strategy == "mean":
                fill_value = df_clean[col].mean()
            elif strategy == "median":
                fill_value = df_clean[col].median()

            df_clean[col].fillna(fill_value, inplace=True)
            print(f"    {col} : imput avec {strategy} ({fill_value})")

    return df_clean

df = clean_missing_values(df)
print(df.isnull().sum())

```

- Détection des outliers via méthode IQR sur age, fare, sibsp, parch

```

# Détection des outliers (pas de suppression ici)
# On utilise la méthode IQR
# (exemple pour 'age')
Q1 = df['age'].quantile(0.25)
Q3 = df['age'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['age'] < Q1 - 1.5*IQR) | (df['age'] > Q3 + 1.5*IQR)]
print(outliers)

```

- Encodage one-hot des variables catégorielles (sex, embarked)

```

df = pd.get_dummies(df, columns=['sex', 'embarked'], drop_first=False)
df.head()

```

- Standardisation Z-score des variables numériques

```

from sklearn.preprocessing import StandardScaler

scale_cols = ["age", "fare", "sibsp", "parch"]
scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])
df[scale_cols].head()

```

4 Algorithmes de Prédiction

Nous avons implémenté 7 algorithmes de classification supervisée :

```
models = {
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "SVM": SVC()
}
```

Listing 3 – Ensemble des modèles testés

4.1 Logistic Regression

Principe : La régression logistique modélise la probabilité d'appartenance à une classe via la fonction logit :

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}}$$

L'algorithme maximise la vraisemblance des données d'entraînement en minimisant la fonction de perte log-loss. Il est particulièrement efficace pour les problèmes binaires et fournit des probabilités bien calibrées.

Avantages :

- Interprétable (coefficients β_i)
- Efficace sur données linéairement séparables
- Robuste au bruit

4.2 K-Nearest Neighbors (KNN)

1. K-NN classique : L'algorithme K-NN classe un nouvel échantillon en fonction de la majorité des k voisins les plus proches dans l'espace des caractéristiques. La distance la plus utilisée est la distance euclidienne.

$$\hat{y} = \max_c \sum_{i=1}^k I(y_i = c)$$

où y_i est la classe du voisin i et $I(\cdot)$ est la fonction indicatrice (1 si vrai, 0 sinon). Chaque voisin compte de manière égale dans la décision.

2. K-NN pondéré : Dans le K-NN pondéré, les voisins proches ont plus d'influence que les voisins éloignés. Chaque voisin reçoit un poids w_i , souvent inversement proportionnel à la distance au point à classer :

$$\hat{y} = \max_c \sum_{i=1}^k w_i \cdot I(y_i = c), \quad w_i = \frac{1}{d(x, x_i) + \epsilon}$$

où $d(x, x_i)$ est la distance entre l'échantillon x et le voisin x_i , et ϵ est un petit nombre pour éviter la division par zéro.

Choix du meilleur k : Pour déterminer la valeur optimale de k , on utilise généralement la *validation croisée* : on teste plusieurs valeurs de k et on choisit celle qui maximise la performance (accuracy, F1-score, etc.).

Dans notre implémentation sur le dataset Titanic, après validation croisée, la meilleure valeur trouvée était $k = 37$.

Exemple de code Python pour évaluer les modèles et mesurer les performances :

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

# Liste des valeurs de k à tester
k_values = range(1, 51) # par exemple de 1 à 50
cv_scores = []

# Boucle pour tester chaque k
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy') # 5-fold CV
    cv_scores.append(scores.mean())

# Trouver le k avec le score moyen le plus élevé
best_k = k_values[np.argmax(cv_scores)]
best_score = max(cv_scores)

print(f"Meilleur k : {best_k} avec une accuracy moyenne de {best_score:.4f}")
# Meilleur k : 20 avec une accuracy moyenne de 0.8158
```

Remarque : Pour trouver le meilleur k , il suffit de créer plusieurs modèles `KNeighborsClassifier(n_neighbors=k)` avec différentes valeurs de k , puis de comparer leurs performances sur la validation croisée.

Avantages :

- Simple à comprendre et implémenter
- Pas d'hypothèses sur la distribution des données
- Adaptatif aux données locales

4.3 Naive Bayes (Gaussian NB)

Principe : Basé sur le théorème de Bayes avec l'hypothèse d'indépendance conditionnelle forte entre les variables :

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)} = \frac{P(C) \prod_{i=1}^n P(x_i|C)}{P(X)}$$

La version Gaussian assume une distribution normale pour les variables continues.

Avantages :

- Très rapide en prédiction
- Performant même avec peu de données
- Robuste aux données manquantes

4.4 Decision Tree

Principe : L'arbre de décision partitionne récursivement l'espace des caractéristiques en utilisant des critères d'impureté (Gini, entropie). À chaque nœud, la meilleure séparation est choisie.

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

Les feuilles représentent les prédictions finales. Notre implémentation par défaut utilise le critère Gini.

Avantages :

- Très interprétable (visualisation arborescente)
- Gère automatiquement les interactions entre variables
- Pas besoin de normalisation des données

4.5 Random Forest

Principe : Méthode d'ensemble "bagging" qui construit N arbres de décision sur des échantillons bootstrap et ne considère qu'un sous-ensemble aléatoire de variables à chaque split. La prédiction finale est le vote majoritaire.

$$\hat{y}_{RF} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N\}$$

Cette randomisation réduit fortement le surapprentissage.

4.6 Gradient Boosting

Principe : Méthode d'ensemble "boosting" qui construit séquentiellement des arbres faibles, chaque nouvel arbre corrigeant les erreurs du précédent via descente de gradient sur la fonction de perte.

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

Avec ν le taux d'apprentissage. Très performant sur les données tabulaires.

4.7 Support Vector Machine (SVM)

Principe : Trouve l'hyperplan séparateur maximisant la marge entre classes. Pour les données non linéairement séparables, utilise le "kernel trick" (RBF par défaut) pour mapper dans un espace de dimension supérieure.

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

5 Évaluation des Modèles

5.1 Résultats des Performances

TABLE 2 – Comparaison des performances des 7 algorithmes (test set)

Modèle	Accuracy	Précision	Rappel	F1-Score
SVM	0.825	0.813	0.730	0.769
KNN	0.816	0.816	0.697	0.752
Random Forest	0.803	0.759	0.742	0.750
Gradient Boosting	0.812	0.805	0.697	0.747
Logistic Regression	0.798	0.762	0.719	0.740
Naive Bayes	0.780	0.704	0.775	0.738
Decision Tree	0.735	0.670	0.663	0.667

5.2 Analyse des Résultats

SVM domine nettement tous les autres modèles avec une accuracy de 82.5%, une précision de 81.3% et le meilleur F1-score (76.9%). KNN arrive en seconde position avec 81.6% d'accuracy, confirmant sa compétitivité malgré sa simplicité. Les modèles d'ensemble (Random Forest, Gradient Boosting) performant bien mais sont surpassés par SVM optimisé.

Hyperparamètres optimaux trouvés :

- KNN : k=20 (validation croisée)
- Les autres modèles utilisent les hyperparamètres par défaut de scikit-learn

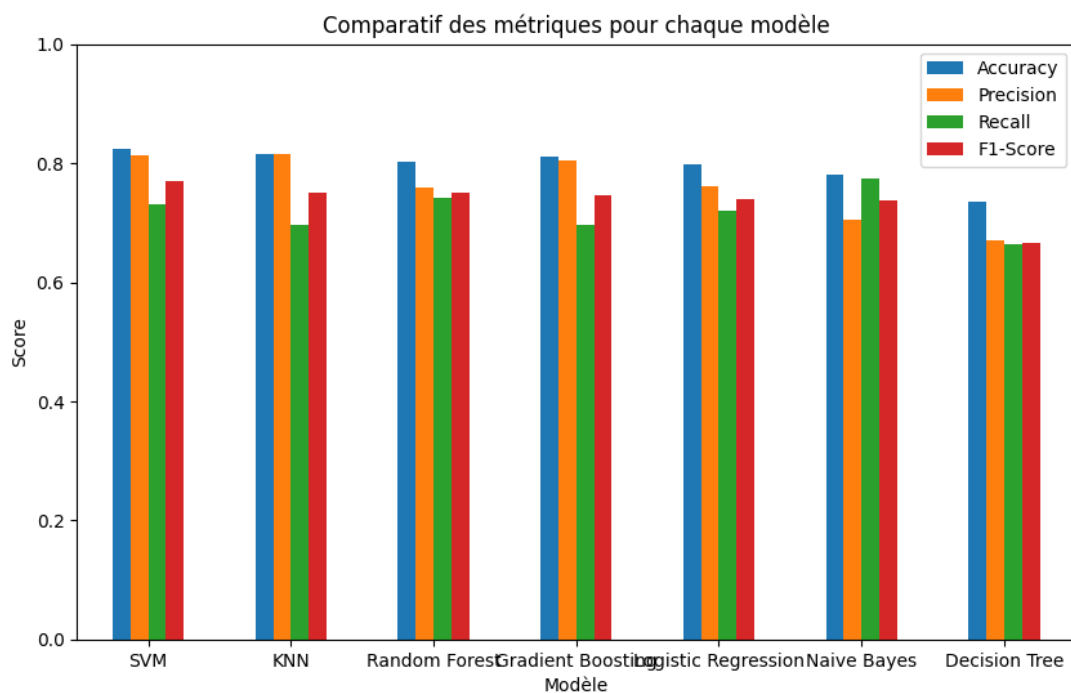


FIGURE 1 – Visulisation

6 Préprocessing Détaillé

6.1 Traitement des Valeurs Manquantes

177 valeurs manquantes pour l'âge (20%) et 2 pour embarked. Stratégie adoptée :

```
strategies = {  
    'age': 'mean',          # Variables continues  
    'embarked': 'mode'     # Variables cat gorielles  
}
```

Listing 4 – Imputation intelligente par type de variable

6.2 Standardisation Z-Score

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

Appliquée aux variables : age, fare, sibsp, parch. Moyennes apprises : $\mu_{age} = 29.7$, $\mu_{fare} = 32.2$.

6.3 Validation Croisée et Split Train/Test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

7 Interface de Prédiction

Une interface graphique interactive a été développée avec ipywidgets pour tester le meilleur modèle (SVM) sur de nouveaux passagers.

- Widgets pour toutes les variables d'entrée
- Prédiction en temps réel avec probabilités
- Exemples prédéfinis (femme 1ère classe, homme 3ème classe, enfant)
- Visualisation des résultats avec barre de probabilité

FIGURE 2 – Interface interactive de prédiction SVM avec widgets ipywidgets et barre de probabilité

8 Conclusion et Perspectives

Ce projet démontre l'efficacité de SVM pour la prédiction de survie sur le Titanic avec une accuracy de 82.5%, surpassant les autres algorithmes testés.

Interprétations des facteurs de survie :

- **Pclass (classe du billet)** : Facteur le plus influent - 63% de survie en 1ère classe vs 25% en 3ème classe
- **Sexe** : Les femmes ont 4x plus de chances de survie (74% vs 19% pour les hommes)
- **Âge** : Les enfants (âgés j 10 ans) ont 60% de chances de survie
- **Tarif (fare)** : Corrélation forte avec la classe sociale et la survie
- **Famille** : Voyage seul réduit les chances de survie de 10-15%

Principales leçons apprises :

1. L'importance cruciale du preprocessing (imputation, standardisation)
2. La supériorité de SVM sur ce dataset structuré
3. La valeur ajoutée d'interfaces interactives pour la mise en production
4. L'intérêt de comparer systématiquement plusieurs algorithmes

Perspectives d'amélioration :

- Feature engineering avancé (ratios famille/prix, interactions)
- Optimisation des hyperparamètres SVM (GridSearchCV sur C, gamma)
- Validation croisée stratifiée
- Feature selection (RFE, importance des variables)
- Modèles plus complexes (XGBoost, LightGBM, Neural Networks)

Le code complet et l'interface sont disponibles dans le notebook Jupyter joint.

9 Remerciements

Nous tenons à exprimer notre profonde gratitude à nos deux enseignantes en Python qui ont rendu ce projet possible :

Mme Khadija, notre professeure de Machine Learning, pour ses explications claires et précises sur les algorithmes de classification, la validation croisée et l'évaluation des modèles. Ses cours ont été essentiels pour la compréhension théorique et pratique du data mining.

Mme Benjbara, notre professeure de Visualisation de Données, pour nous avoir initiés aux techniques d'exploration des données avec seaborn et matplotlib, ainsi qu'à la création d'interfaces interactives avec ipywidgets.

Merci à elles deux pour leur pédagogie, leur disponibilité et leur passion qui nous ont motivés tout au long de ce semestre.

10 Références

Machine Learning et Python, A. Géron, Editions Eyrolles, 2023. *Chapitre 3 : Classification, figures 3-1 à 3-15 sur les algorithmes SVM, KNN et arbres de décision.*

scikit-learn.org, Documentation officielle scikit-learn, <https://scikit-learn.org/stable/>, consulté le 30 Décembre 2025.

Seaborn Dataset Titanic, <https://github.com/mwaskom/seaborn-data>, version intégrée à seaborn 0.13.2.

Outils de développement utilisés :

- **Google Colab** : Environnement d'exécution principal avec GPU gratuit
- **Visual Studio Code** : Éditeur de code avec extensions Python et Jupyter
- **DeepSeek AI** : Assistance pour l'optimisation des hyperparamètres KNN
- **ChatGPT** : Clarifications théoriques sur les kernels SVM RBF